

Policy RAG Chatbot — Project Plan

Colab/Kaggle-friendly implementation plan for a multi-organization policy/document assistant

Scope decision: The initial implementation will support PDF policies. DOCX, TXT/Markdown, and Web URL ingestion are future extensions. Conversation history is intentionally moved to **Future Enhancements** for the first version.

1. Project Objective

Build a small but industry-oriented RAG chatbot that answers questions strictly from approved policy/document content. The system must support different companies, organizations, or institutes and prevent information from one organization from being used to answer questions about another.

2. Confirmed Requirements

Area	Decision
Initial document format	PDF
Future document formats	DOCX, TXT/Markdown, Web pages/URLs
Vector database	Free long-term option; Qdrant Cloud is the planned choice
Organizations	Multiple companies / organizations / institutes
Policy updates	Latest policy only for normal retrieval
Version detection	Effective date, version/date in content, filename; fallback if uncertain
Missing answer	"I couldn't find this information in the available policies."
Question scope	Policy/document questions only
Testing controls	Organization and LLM model via dropdowns
Conversation history	Future enhancement, not part of initial implementation

3. Target UI — Initial Version

The first notebook UI should be a simple Gradio application suitable for Google Colab/Kaggle. It should feel like a policy assistant rather than a generic PDF chatbot.

UI area	Initial behavior
Organization dropdown	User selects Company A, Company B, Institute A, etc. Testing control only.
LLM dropdown	User selects an available configured model. Testing/model-comparison control only.
PDF upload	Upload a policy PDF for the selected organization.
Policy metadata	Show detected policy name, version, effective date, and detection status.
Latest-policy status	Show whether the system confidently identified the latest policy or requires manual fallback.
Chat box	User asks a policy/document question.
Answer	Strict answer generated only from retrieved policy context.
Citation	Every supported answer shows policy name/version and page/section when available.
No-answer behavior	Show the exact required fallback message when the information is not available.

4. Logical UI Layout

Header: Policy Assistant
Controls: Organization ▼ | LLM Model ▼ | Upload PDF
Policy status: Active/latest policy information and version-detection result
Chat: User question → retrieved context → strict answer
Source area: Policy name, version, effective date, page/section

5. System Architecture

PDF Upload → PDF Text Extraction → Policy Metadata Detection → Version Resolution → Chunking → Embeddings → Qdrant Cloud → Organization + Latest-Policy Filtering → Semantic Retrieval → Context Builder → Strict LLM → Answer + Citations

6. Multi-Organization Design

Every indexed chunk will carry organization and policy metadata. Retrieval must filter by the organization selected in the UI before semantic search results are used.

Example metadata:
organization = Company_A
policy_name = Leave Policy
policy_version = v3.0
effective_date = 2026-01-01
source_file = company_a_leave_policy.pdf
page = 12
section = Annual Leave

7. Latest Policy Resolution

The system will detect the latest policy using multiple signals rather than relying on one naming convention.

Priority	Signal
1	Explicit effective date in the policy content
2	Version number/date in policy content
3	Version/date detected from filename
4	If signals conflict or are insufficient: mark as unresolved and require manual selection

Safety rule: The system must not silently choose an uncertain policy as the latest policy. Only a confidently resolved latest/active policy participates in normal retrieval.

8. Strict RAG Answering Rules

- Answer only from retrieved policy/document context.
- Do not use general LLM knowledge to fill missing policy information.
- Do not infer a company-specific rule when the document does not state it.
- Every answer must include supporting citation information.
- If the information cannot be supported by available policies, return: "I couldn't find this information in the available policies."
- Non-policy/general questions should be rejected as outside the chatbot scope.

9. Citation Design

Citations should come from retrieved chunk metadata, not be invented by the LLM. Where available, display policy name, version, effective date, source filename, page number, and section.

Example:

Source: Company A — Leave Policy v3.0

Effective: 01-Jan-2026

Page: 12

Section: Annual Leave

10. Initial Implementation Phases

Phase	Work
1	Refactor the existing workshop notebook and preserve useful Gemini/RAG learning sections.
2	Implement PDF upload and page-aware text extraction.
3	Implement policy metadata extraction and version/date detection.
4	Add multi-organization metadata and organization-filtered retrieval.
5	Connect embeddings and Qdrant Cloud for persistent vector storage.
6	Implement latest-policy-only retrieval with manual fallback.
7	Implement strict policy-only RAG prompt and required no-answer behavior.
8	Build Gradio testing UI with organization and LLM dropdowns.
9	Test organization isolation, version conflicts, missing answers, citations, and follow-up questions.

11. Future Enhancements

The following features are intentionally **not part of Version 1**. They can be added after the core PDF → retrieval → answer pipeline is stable.

Future feature	Planned capability
Conversation history	Persist conversations and messages so users can reopen previous chats.
History sidebar	Show chats grouped by Today, Yesterday, and older dates.
New Chat / Rename / Delete	Manage saved conversations.
Conversation search	Search previous questions and answers.
Contextual follow-up	Use recent conversation history to resolve references such as “that policy” or “does that include...?” while
Chat metadata	Store organization, model, policy version, citations, timestamps, and message-level sources.
History database	Use SQLite initially; migrate to a hosted relational database when needed.
Policy comparison	Compare current and previous policy versions.
Additional ingestion	DOCX, TXT/Markdown, and Web URL sources.
Authentication / roles	Separate administrator policy management from normal user chat.
Evaluation dashboard	Track retrieval quality, citation correctness, answer faithfulness, and unanswered questions.

12. Future Conversation History Data Model

When history is implemented, keep it separate from the vector database. Qdrant stores policy knowledge and embeddings; a relational database stores application/conversation data.

Suggested entities:
organizations
policies
conversations
messages
message_sources

A message record should retain the user question, assistant response, model used, timestamp, organization, and source/citation references. This makes history auditable and allows a previous conversation to be reopened without treating chat history itself as policy knowledge.

13. Recommended Version-1 Boundary

Build now: PDF ingestion, multiple organizations, latest-policy detection, Qdrant persistence, strict policy RAG, citations, no-answer behavior, and testing dropdowns.

Do later: conversation history, history search, chat management, policy comparison, additional document formats, authentication, and evaluation dashboards.

14. Success Criteria

- ✓ A PDF can be uploaded and indexed for a selected organization.
- ✓ The system can identify the latest policy using supported metadata signals or safely request manual fallback.
- ✓ A user can select an organization and retrieve only that organization's active/latest policy content.
- ✓ Answers are grounded only in retrieved policy context.
- ✓ Every supported answer contains a citation.

- ✓ Unsupported questions return the required fallback message.
- ✓ The same RAG pipeline can be tested with different configured LLMs.
- ✓ The notebook runs in a Google Colab/Kaggle-friendly environment without requiring a local GPU.